ware Engineering

FE101

Anonymous functions

Today

Welcome to your next lesson: Anonymous functions.

I'm preparing your personalized lesson based on what you've already learned, what's coming up next, and the best way to build your skills.

Step by step, you’re moving forward.

You just learned that in JS we can treat functions like values (store them in variables, pass them around). Now we’ll do the same thing but without giving the function a name — that’s an anonymous function 🔍

First, let’s connect to what you already know.

In JavaScript, you’ve seen a named function expression like:

“Make a variable and store a function in it.”

Visually:

Text
[variable name] = [function name] (...) { ... }
But today’s pattern is:

Text
[variable name] = function (...) { ... }
^
no function name here
So the only new thing is:

we remove the function’s own name after function.

Let’s start super small:

Open the JS code editor for this lesson.
Create a named function expression first:
Javascript
```
let greet = function greetFunction() {
    console.log("Hello from a named function expression");
};

greet();
```
Run it, then Submit code so I can see it.

We’ll turn this into an anonymous function together next.

Javascript
```
let greet = function greetFunction() {
    console.log("Hello from a named function expression");
};

greet();
```

Nice, Ron — that’s exactly the named function expression I was hoping to see 🎯

Right now you have:

Javascript
```
let greet = function greetFunction() {
    console.log("Hello from a named function expression");
};

greet();
```
Step by step, let’s turn this into an anonymous function:

In the editor, change the line

```
let greet = function greetFunction() {
```
to

```
let greet = function() {
```
(just delete greetFunction and keep everything else the same)

Run it, then Submit code again.

After you do that, tell me: does the output change or stay the same?

Javascript
```
let greet = function() {
    console.log("Hello from a named function expression");
};

greet();
```
Perfect 👌

You now have an anonymous function:

Javascript
```
let greet = function() {
    console.log("Hello from a named function expression");
};

greet();
```
The function still works the same, but the function itself has no name after function. That function() { ... } shape is what I want you to recognize on sight.

Quick check-in 🧠

When you see this:

Javascript
```
function() {
    console.log("Hi");
}
```
do you recognize that as:

A) A normal named function

B) An anonymous function (no name)

Just answer A or B in your own words, then we’ll move to using these for one-time actions.

b

Exactly — B is right 🎯

function() { ... } with no name is an anonymous function.

Next step: use one for a one-time action.

I want you to do this in the editor:

Delete your old code.
Add this shape:
Javascript
```
let sayOnce = function() {
    console.log("This runs only once");
};

sayOnce();
```
Type it yourself (don’t paste), then Run and Submit code.

We’ll tweak this after you submit.

Javascript
```
let sayItOnce = function() {
    console.log("This will only run once");
};

sayItOnce();
```
Nice adjustment, Ron ☑

You just wrote and used an anonymous function for a one-time use:

Javascript
```
let sayItOnce = function() {
    console.log("This will only run once");
};

sayItOnce();
```
Here the anonymous function() { ... } is stored in sayItOnce, and then you call it once.

Next step: timers ⏱

I want to show how we use an anonymous function as a callback with setTimeout.

In the editor, replace your code with this:

Javascript
```
setTimeout(function() {
    console.log("This runs after 2 seconds");
}, 2000);
```
Then Run and Submit code.

Don’t worry about remembering everything yet — just get this working.

Javascript
```
setTimeout(function() {
    console.log("This runs after 2 seconds");
}, 2000);
```
Nice — that’s exactly it 🎯

You just used an anonymous function as a callback to setTimeout:

Javascript
```
setTimeout(function() {
    console.log("This runs after 2 seconds");
```

}, 2000);
Visually:

setTimeout([anonymous function] , 2000);
The anonymous function() { ... } is saved by setTimeout
Then it’s called later (after 2000 ms)
Quick understanding check (one simple question):

In your own words:

Does the anonymous function run immediately when setTimeout is called, or does it run later after the delay?

You can just answer “immediately” or “later”.

after delay the interpreter is even fooled ito saying your program ran without any output then 2 seconds later it prints

Love that observation 😄 Exactly: the program finishes from the interpreter’s point of view, then the timer’s callback fires later and prints.

So you’ve now:

Written an anonymous function() { ... }
Used it for a one-time action
Used it as a timer callback with setTimeout
Last tiny skill: visually spotting anonymous function syntax.

When you look at this line:

Javascript
```
setTimeout(function() {
    console.log("Hi");
}, 1000);
```
can you point to just the anonymous function part and rewrite only that part here in chat?

You don’t need the whole setTimeout line — just the function() { ... } bit as you see it.

```
function() {
console.log("Hi");
}
```

Exactly right 🎯

This part:

Javascript
```
function() {
  console.log("Hi");
}
```
is an anonymous function — it’s a full function, just without a name after function.

So at this point you can:

Recognize function() { ... } as an anonymous function
Write your own for one-time use
Use one as a callback with setTimeout
Do you want to do one more quick practice with an anonymous function and setTimeout, or

are you feeling solid and ready to move on from this topic?

I’m ready to move on

Nice work, Ron 🎯

You’ve got the essentials of anonymous functions down:

You can spot the function() { ... } pattern as an anonymous function.
You can write unnamed functions for one-time use.
You can pass anonymous functions into setTimeout as callbacks that run later.

This sets you up really well for all the callback-heavy JS you’ll see next.

As you go forward, keep an eye out for any time you see:


```
function() {
    // ...
}
```

inside another function call — that’s your anonymous callback in the wild 👀

When you’re ready, hit Next lesson in the Learn tab and we’ll keep building from here 🚀


```
setTimeout(function() {
    console.log("This runs after 2 seconds");
}, 2000);
```